

PS0 — Introduction & Installation (Environment Setup)

Source: MIT OpenCourseWare, 6.100L *Introduction to CS and Programming Using Python*, Fall 2022, Problem Set 0. Adapted for this course under the Creative Commons **CC BY-NC-SA 4.0** license (<https://ocw.mit.edu/terms>). This adaptation is likewise shared under CC BY-NC-SA 4.0.

1) Assignment Overview

Item	Value
Original assignment	MIT 6.100L Fall 2022, Problem Set 0: Introduction & Installation
Topic	Development environment setup; first Python program
Assigned week	Week 1
Grading	Ungraded (completion check only — but required before PS1)
Environment for this course	VS Code + Python 3 on macOS or Windows (replaces the original Anaconda/Spyder setup)

Objectives

- Install Python 3 and VS Code on your own machine and verify the installation.
- Run your first Python program (variables, arithmetic, importing a library).
- Learn the hand-in procedure used for every problem set in this course.

Note (course adaptation). The original PS0 also contained an MIT-internal background survey, a "Pset Buddy" sign-up, and a class-logistics quiz. Those items are replaced here by our own course-orientation form distributed separately in class.

2) Installation Guide

This course uses **Python version 3.10 or higher**. Python 3 is not backwards compatible with Python 2.x, so make sure the version number displayed starts with **3.**

2.1) Install Python 3

macOS

1. Go to <https://www.python.org/downloads/> and download the latest Python 3 installer for macOS.
2. Run the downloaded `.pkg` file and follow the installer with default options.
3. Open **Terminal** (Spotlight → type "Terminal") and verify:

```
python3 --version
```

You should see something like `Python 3.12.x`.

Windows

1. Go to <https://www.python.org/downloads/> and download the latest Python 3 installer for Windows (64-bit).
2. Run the installer. **IMPORTANT: check the box "Add python.exe to PATH"** on the first screen before clicking Install.
3. Open **Command Prompt** (or PowerShell) and verify:

```
python --version
```

You should see something like `Python 3.12.x`.

2.2) Install VS Code

macOS

1. Download VS Code from <https://code.visualstudio.com/> (Universal build).
2. Unzip and drag **Visual Studio Code.app** into the **Applications** folder, then launch it.

Windows

1. Download the VS Code installer from <https://code.visualstudio.com/>.
2. Run the installer with default options (keep "Add to PATH" checked).

2.3) Install the Python extension for VS Code

1. Launch VS Code.
2. Open the Extensions view (macOS: `Cmd+Shift+X` / Windows: `Ctrl+Shift+X`).
3. Search for **"Python"** and install the extension published by **Microsoft** (this also installs Pylance).
4. Restart VS Code.

2.4) Install numpy

Open the terminal (in VS Code: menu **Terminal** → **New Terminal**) and run:

- macOS: `python3 -m pip install numpy`
- Windows: `python -m pip install numpy`

2.5) Create your course folder

Create a folder such as `CTC/ps0` (avoid Korean characters and spaces in the path if possible), and in VS Code use **File** → **Open Folder...** to open it. All problem-set files this semester should live under this course folder.

3) Problem — Your First Program: Raising a Number to a Power and Taking a Logarithm

(This is the original PS0 programming exercise, preserved unchanged.)

The goal of this exercise is to make sure your Python and numpy installations are correct, to get you comfortable with your editor, and to begin using simple elements of Python: printing results (`print`), reading user input (`input`), and storing values in variables.

Assignment. Write a program that does the following **in order**:

1. At the top of your file, type: `import numpy`
2. Now write a line that sets a variable named `x` to 5.
3. Now write a line that sets a variable named `y` to 8.
4. Add variables `x` and `y`, and save the result to a variable named `z`.
5. Now save the result of this command: `numpy.log2(z)` to a variable named `a`.

Use VS Code to create your program, and save your code in a file named `ps0.py`.

Hints:

- Remember that if you want to hold onto a value, you need to store it in a *variable* (i.e., give it a name to which you can refer when you want that value).

3.1) Verification script

Add the following two lines at the very end of `ps0.py`, run the file in VS Code (► "Run Python File" button, or `python3 ps0.py` in the terminal), and take a **screenshot** of the editor window showing both your code and the output:

```
print("z =", z, "/ a =", a)
print("Python OK, numpy OK")
```

Expected output:

```
z = 13 / a = 3.700439718141092
Python OK, numpy OK
```

4) Deliverables

Submit the following to the course LMS by the deadline announced in class:

1. `ps0.py` — your program from Section 3.
2. **Screenshot (PNG or JPG)** — VS Code window showing (a) your code, (b) the terminal output of the verification script, and (c) the Python version (`python3 --version` output) visible in the terminal.
3. At the start of `ps0.py`, in comments, write your name, student ID, approximate time spent, and collaborators (if any):

```
# Problem Set 0
# Name: Hong Gildong
# Student ID: 20261234
# Collaborators: none
# Time Spent: 1:30
```

5) Grading Rubric Summary

PS0 is **ungraded** (pass/fail completion check). For reference, all graded problem sets (PS1–PS5) use the common course rubric:

Criterion	Weight
Correctness & edge cases	45%
Problem decomposition (functions, structure)	20%
Student-written tests	15%
Explanation (comments / docstrings / write-up)	10%
Code clarity & style	10%

6) [Instructor Checklist]

Changes from the original (MIT 6.100L F22 PS0):

1. Background Survey (§1), Pset Buddy (§2), Class Info Assessment (§3) — MIT-internal items **removed**, replaced by our course orientation form.
2. Environment changed from **Anaconda + Spyder** to **VS Code + Python 3** (per course design document); installation guide rewritten accordingly for macOS and Windows.
3. Hand-in changed from MIT website upload to course LMS upload; **verification screenshot** requirement added.
4. Programming exercise (§4.3 of the original, the `ps0.py` numpy program) — **preserved unchanged**; a 2-line verification print block was appended for the screenshot evidence.

Problem count: original programming problems 1 → packet programming problems 1 (equal).

Prerequisite check: ☒ Confirmed — no items beyond Week 1 prerequisites (installation, variables, arithmetic, `import`, `print` only; `numpy.log2` is used as a black box exactly as in the original).